Guidelines for Grade 9

Chủ đề #6:

CẤU TRÚC DỮ LIỆU

Các hàm của deque tương tự queue, nhưng có thêm 2 hàm để thao tác với hai đầu của hàng đợi:HĐã bao giờ bạn gặp một bài toán khó, bạn nghĩ ra được giải thuật, đã thử với rất nhiều test và luôn đúng nhưng lại không biết code như thế nào? Đó là do bạn đang gặp vấn đề về lưu trữ, truy cập và xử lý dữ liệu. Hãy cùng GG9 giải quyết vấn đề này nhé!

I. VECTOR

1. Khái niệm:

Vector là một mảng động lưu trữ một dãy giá trị có cùng kiểu dữ liệu. Các tính chất cơ bản của vector tương đối giống với array (mảng thông thường). Nhưng vector có các khả năng linh hoạt hơn như: thêm, xóa, thay đổi kích thước, thay đổi một (hoặc nhiều) phần tử trong mảng,...

2. Cài đặt:

a. Khai báo:

- Vector rỗng: ***vector <kiểu_dữ_liệu> tên_vector***.
  Ví dụ: `vector <int> v;`
- Vector kích thước N: ***vector <kiểu_dữ_liệu> tên_vector (N)***.
  Ví dụ: `vector <int> v1(5); vector <int> v2(N);`
- Vector với N phần tử có gán giá trị khởi đầu:
  vector <kiểu_dữ_liệu> tên_vector (N) = {giá_trị_1, giá_trị_2, ..., giá_trị_N}.
  Ví dụ: `vector <int> v(5) = {1, 2, 3, 4, 5};`
- Vector với N phần tử có gán giá trị khởi đầu *giống nhau*:
  vector <kiểu_dữ_liệu> tên_vector (N, giá_trị)

Ví dụ: `vector <int> v(5, 100);`

- Vector dạng mảng 2 chiều: ***vector <vector <kiểu_dữ_liệu>> tên_vector***.
  Ví dụ: `vector <vector <int>> v;`
- Vector dạng mảng 2 chiều có kích thước N*M:
  vector <vector <kiểu_dữ_liệu> tên_vector (N, vector <kiểu_dữ_liệu>(M))
  `vector <vector <int>> v(N, vector <int>(M));`

b. Truy cập:

- Dạng mảng một chiều: ***tên_vector [chỉ_số]*** → `v[i]`
- Dạng mảng hai chiều: ***tên_vector [chỉ_số_dòng][chỉ_số_cột]*** → `v[i][j]`

==> Cách truy cập giống array thông thường. Tuy nhiên, ta có thể thấy rằng với mảng hai chiều, vector được lưu trữ ở dạng lồng nhau. Nếu khai báo nhiều vector lồng nhau hơn, sẽ có các cách truy cập phức tạp hơn.

c. Các hàm cơ bản: *(Giả định các vector tên **v** và có **n** phần tử)*

Hàm	Tác dụng	Cài đặt
`size()`	Trả về kích thước của vector → **O(1)**	`v.size();`
`empty()`	Kiểm tra vector có rỗng không? → **O(1)**	`v.empty();` `!v.empty();`
`begin()`	Trả về con trỏ vị trí đầu tiên của vector. → **O(1)**	`v.begin();`
`end()`	Trả về con trỏ vị trí cuối cùng của vector. → **O(1)**	`v.end();`

`sort()`	• Sắp xếp các phần tử trong đoạn [i; j] tăng dần.
	• Kết hợp điều kiện ***greater*** sắp xếp các phần tử trong đoạn [i; j] giảm dần.
	→ **O(n * log(n))**
		`sort (v.begin()+i, v.begin()+j);`
		`//----------`
		`sort (v.begin()+i, v.begin()+j, greater <int> ());`
		`// sort cả vector:`
		`sort (v.begin(), v.begin() + n);`

`push_back()`	Thêm một phần tử ***x*** có cùng KDL vào cuối vector → **O(1) đến O(n)**	`v.push_back(x);`
`pop_back()`	Xóa phần tử cuối của vector → **O(1)**	`v.pop_back();`
`clear()`	Xóa tất cả phần tử trong vector → **O(n)**	`v.clear();`
	• Thêm phần tử ***x*** vào vị trí ***i*** → **O(n)**	`v.insert(v.begin() + i, x);` `//----------`

`insert()`	• Thêm ***m*** phần tử vào vị trí ***i***, ta tạo vector chứa ***m*** phần tử đó rồi thêm vector đó vào ***v*** → **O(n + m)**	`vector <int> ins(m) = {...};`
		
		`v.insert(v.begin() + i, ins.begin(), ins.end());`

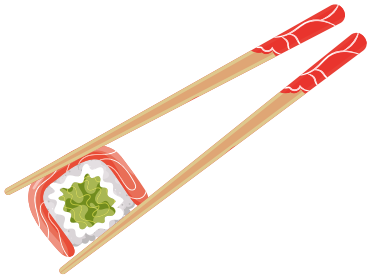
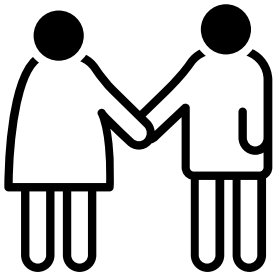
`erase()`	• Xóa phần tử ở vị trí ***i*** → **O(n)**
	• Xóa m phần tử tính từ vị trí ***i*** → **O(n + m)**
		`v.erase(v.begin() + i);`
		
		`//--------------------`
		
		`v.erase(v.begin() + i, v.begin() + i + m + 1);`

`find()`	Tìm phần tử ***x*** ∈ ***[L; R]*** với ***1*** ⩽ ***L*** ⩽ ***R*** ⩽ ***n*** trong vector → **O(n)**
	
	Nếu tìm thấy, chương trình trả về vị trí của ***x***, ngược lại chương trình trả về ***v.end()***
		`auto it = find(v.begin(), v.end(), x);`
		
		`if (it != v.end()) {`
		`  // code (tìm thấy x)`
		`} else {`
		`  // code (không thấy x)`
		`}`

II. PAIR

1. Khái niệm:

Pair là một kiểu container lưu trữ dữ liệu theo cặp. Được sử dụng để kết hợp hai kiểu dữ liệu với nhau, nó cung cấp cách lưu trữ 2 giá trị đi kèm nhau nhưng chỉ sử dụng 1 biến. Việc sử dụng pair sẽ tiện lợi hơn hai biến riêng biệt khi ta cần có sự liên kết giữa hai biến thành phần. Tương tự, xung quanh ta cũng có những thứ phải có đôi, có cặp thì mới hiệu quả.

2. Cài đặt:

a. Khai báo:

- Pair đơn rỗng: ***pair*** <***kiểu_dữ_liệu_1, kiểu_dữ_liệu_2***> ***tên_pair***;
  Vd: `pair <int, int> p;`
- Mảng có ***N*** phần tử Pair: ***pair*** <***kiểu_dữ_liệu_1***, ***kiểu_dữ_liệu_2***> ***tên_pair*** [***N***]
  Vd: `pair <int, int> p[10];`
- Pair đơn có cặp phần tử ban đầu:
  pair <***kiểu_dữ_liệu_1***, ***kiểu_dữ_liệu_2***> ***tên_pair*** = ***make_pair***(***giá_trị_1***, ***giá_trị_2***)
  Vd: `pair <int, int> p = make_pair(2, 3);`

→ Kiểu dữ liệu của các phần tử trong pair có thể là các kiểu dữ liệu thường gặp như ***int***, ***double***, ***long long***, ***char***... hay cũng có thể chính là ***một pair khác***. Ngoài ra nó còn có thể là các kiểu phức tạp hơn như ***các kiểu dữ liệu lồng nhau***.

b. Truy cập:

- Dạng pair đơn: ***tên_pair***.***first***/***second***
  Vd: `p.first; p.second;`
- Dạng pair mảng: ***tên_pair***[***i***].***first***/***second***
  Vd: `p[i].first; p[i].second;`

c. Các hàm cơ bản:

Hàm	Tác dụng	Cài đặt
`make_pair()`	Ghép hai phần tử có sẵn thành một cặp → **O(1)**	`p = make_pair(x,y);`
`pair.first`	Truy cập phần tử ở khóa đầu tiên → **O(1)**	`p.first`
`pair.second`	Truy cập phần tử ở khóa thứ hai→ **O(1)**	`p.second`
`swap(x, y)`	Hoán đổi phần tử của hai cặp → **O(1)**	`swap(x, y);`

`sort()`	• Sắp xếp các cặp phần tử trong đoạn [i; j] tăng dần.
	• Kết hợp điều kiện ***greater*** sắp xếp các cặp phần tử trong đoạn [i; j] giảm dần.
	→ **O(n * log(n))**
		`sort(p + i, p + j);`
		`//----------`
		`sort(p+i, p+j, greater<int>());`
		`// sort theo giá trị thứ hai:`
		`bool cmp(const pair<int, int>& a, const pair<int, int>& b) {`
		`return a.second < b.second;`
		`// So sánh phần tử thuộc khóa thứ hai`
		`sort(p, p+n, cmp);`

III. SET

1. Khái niệm:

Set là CTDL lưu trữ các phần tử ***khác nhau*** được sắp xếp ***tăng dần***. Set có lợi thế có thể chèn, gỡ hoặc tìm kiếm phần tử với độ phức tạp ***O(log n)***. Nhược điểm cũng như là khác biệt với vector đó là không thể truy cập trực tiếp phần tử nằm ở vị trí bất kỳ.

2. Cài đặt:

a. Khai báo:

- Set rỗng: **set <*kiểu_dữ_liệu*> *tên_set***.
  Ví dụ: `set <int> s;`
- Set có một số phần tử ban đầu:
  set <kiểu_dữ_liệu> tên_set = {giá_trị_1, giá_trị_2, ..., giá_trị_N}.
  Ví dụ: `set <int> s = {3, 4, 1, 2, 4};`
  Lưu ý: Vì set sẽ không lưu các phần tử trùng lặp và các phần tử của set được sắp xếp tăng dần nên ***s*** sau cùng sẽ có các phần tử `{1, 2, 3, 4}`

Set với kiểu so sánh khác: Khai báo ***set <int> s*** trong ví dụ sử dụng kiểu so sánh bé hơn **(<)**, tức mỗi phần tử đứng trước sẽ nhỏ hơn phần tử phía sau. Có thể sử dụng kiểu so sánh lớn hơn **(>)** với cú pháp ***set<int, greater<int>>***. Khi đó các phần tử đứng trước sẽ lớn hơn các phần tử đứng sau.
→ Ví dụ `set <int, greater<int>> s = {3, 4, 1, 2, 4};` sau khi khởi tạo sẽ có các phần tử là `{4, 3, 2, 1}`

b. Các hàm cơ bản *(Giả định các set tên **s**)*

Hàm	Tác dụng	Cài đặt & ví dụ
`size()`	Trả về kích thước của set → **O(1)**	`s.size();`
`empty()`	Kiểm tra set có rỗng không? Nếu có trả về true, nếu không trả về false → **O(1)**	`bool b = s.empty();` `if(!s.empty()) {...}`
`begin()`	Trả về con trỏ vị trí đầu tiên của set. → **O(1)**	`s.begin();`
`end()`	Trả về con trỏ vị trí cuối cùng của set. → **O(1)**	`s.end();`
`clear()`	Xóa tất cả phần tử trong set→ **O(n)**	`v.clear();`

`insert()`	• Thêm một phần tử ***x*** vào set → **O(log n)**
	• Thêm nhiều phần tử vào set từ con trỏ của CTDL khác
	→ **O(n*log (n+size))**
	Phần tử sẽ được sắp xếp và đưa về vị trị tương ứng
		`v.insert(x);`
		`//--------------------`
		`vector <int> ins(m) = {...};`
		`v.insert(ins.begin(), ins.end());`

	• Xóa phần tử ở vị trí được chỉ bằng con trỏ của set → **O(1)**
	• Xóa phần tử có giá trị là ***x*** → **O(log**
		`set<int>::iterator ite = ..`
		`s.erase(ite);`
		`//--------------------`
		`int val = 109;`

`erase()`	**n)**
	• Xóa các phần tử trong khoảng chỉ bằng con trỏ → **O(n) theo chiều dài cần xóa**
		`s.erase(val);`
		`//----------`
		`set<int>::iterator ite = ..`
		`set<int>::iterator ite2 = .`
		`s.erase(ite, ite2);`

`find()`	Tìm phần tử ***x*** trong set → **O(log n)**
	• Nếu tìm thấy, trả về con trỏ chỉ đến phần tử đó
	• Nếu không, trả về `s.end()` (con trỏ chỉ sau phần tử cuối)
		`set<int> s = {1, 3, 4, 9};`
		`auto it = s.find(3);`
		`if (it != v.end()) {`
		`  // code (tìm thấy x)`
		`} else {`
		`  // code (không thấy x)`
		`}`

`lower_bound()`	Trả về con trỏ chỉ phần tử đầu tiên **không đứng sau *x*** trong set → **O(log n)**
	Ví dụ với set mặc định, trả về phần tử đầu tiên ≤ phần tử ***x***
		`set<int> s = {1, 3, 4, 9};`
		`auto it = s.lower_bound(3);`
		`cout << it-s.begin();  // 3`
		`it = s.lower_bound(5);`
		`cout << it-s.begin();  // 4`

`upper_bound()`	Trả về con trỏ chỉ phần tử đầu tiên **đứng sau *x*** trong set → **O(log n)**
	Ví dụ với set mặc định, trả về phần tử đầu tiên > phần tử ***x***
		`set<int> s = {1, 3, 4, 9};`
		`auto it = s.upper_bound(7);`
		`cout << it-s.begin();  // 9`
		`it = s.upper_bound(4);`
		`cout << it-s.begin();  // 9`

c. Truy cập:

Như đã đề cập, không thể truy cập phần tử ở vị trí bất kỳ trong set. Truy cập duy nhất có thể là sử dụng *s.begin(), tức truy cập giá trị của con trỏ chỉ phần

tử đầu tiên của set. Các phần tử kế đến, phải tăng giá trị con trỏ một cách thủ công, với độ phức tạp **O(n)**

```
// Truy cap phan tu vi tri thu 5 cua set
set <int> s = {1, 3, 4, 5, 6, 9, 12};
...
auto ite = s.begin();
// Lap lai 5 lan
for (int i = 0; i < 5; i++) ite++;
cout << *ite;    // 6
```

Con trỏ của set là kiểu dữ liệu có dạng ***khai_báo_của_set::iterator***. Có thể sử dụng auto để rút gọn. Ví dụ:

```
set<int, greater<int>> myset;

set<int, greater<int>>::iterator ite_myset = myset.begin();
```

d. Mở rộng: Unordered set

Tương tự như set, ***unordered_set*** là CTDL lưu trữ các phần tử khác nhau, cũng không thể truy cập trực tiếp phần tử. Nhưng khác biệt là các phần tử của unordered set không được sắp xếp. Chèn, gỡ hoặc tìm kiếm phần tử vận hành độ phức tạp ***O(k)***, dao động khoảng ***O(1)*** → ***O(n)***. Dù thấy rằng unordered set tối ưu hơn set trong những trường hợp không cần sắp xếp, nó vẫn có thể chậm hơn set trong những trường nhất định

Khai báo:

- Unordered set rỗng: **unordered_set<*kiểu_dữ_liệu*> *tên_set***.
  Ví dụ: `unordered_set<int> s;`
- Unordered set có một số phần tử ban đầu:
  unordered_set<kiểu_dữ_liệu> tên_set = {giá_trị_1, giá_trị_2, ..., giá_trị_N}.
  Ví dụ: `unordered_set<int> s = {3, 4, 1, 2};`

Các hàm cơ bản *(Giả định các **unordered_set** tên **s**)*

Hàm	Tác dụng	Cài đặt & ví dụ
`size()`	Trả về kích thước của unordered set → **O(1)**	`s.size();`
`empty()`	Kiểm tra unordered set có rỗng không? Nếu có trả về ***true***, nếu không trả về ***false***. → **O(1)**	`bool b = s.empty();` `if(!s.empty()) {...}`
`begin()`	Trả về con trỏ vị trí đầu tiên của unordered set. → **O(1)**	`s.begin();`
`end()`	Trả về con trỏ vị trí cuối cùng của unordered set. → **O(1)**	`s.end();`
`clear()`	Xóa tất cả phần tử trong set. → **O(n)**	`v.clear();`

`insert()`	• Thêm một phần tử ***x*** vào unordered set. → **O(1) đến O(n)**
	• Thêm nhiều phần tử vào set từ con trỏ của các CTDL khác. → **O(n) đến O(n*(size+1))**
	
	Phần tử thêm vào không chắc đứng cuối
		`v.insert(x);`
		
		`//--------------------`
		
		`vector <int> ins(m) = {...};`
		
		`v.insert(ins.begin(), ins.end());`

	• Xóa phần tử ở vị trí được chỉ bằng	`unordered_set<int>::iterator ite = ..`
		
		`s.erase(ite);`
		
		`//--------------------`

`erase()`	con trỏ của unordered_set.
	• Xóa phần tử có giá trị là ***x.***
	• Xóa các phần tử trong khoảng chỉ bằng con trỏ.
	
	→ **O(số phần tử xóa) đến O(n)**
		`int val = 109;`
		
		`s.erase(val);`
		
		`//--------------------`
		
		`unordered_set<int>::iterator ite = ..`
		
		`unordered_set<int>::iterator ite2 = .`

`find()`	Tìm phần tử ***x*** trong set → **O(1) đến O(n)**
	
	• Nếu tìm thấy, trả về con trỏ chỉ đến phần tử đó.
	• Nếu không, trả về s.end() (con trỏ chỉ sau phần tử cuối).
		`unordered_set<int> s = {1, 3, 4, 9};`
		
		`auto it = s.find(3);`
		
		`if (it != v.end()) {`
		`  // code (tìm thấy x)`
		`} else {`
		`  // code (không thấy x)`
		`}`

Truy cập:

Tương tự như set, unordered_set không thể truy cập phần tử ở vị trí bất kỳ và phải tăng giá trị con trỏ một cách thủ công, với độ phức tạp **O(n)**.

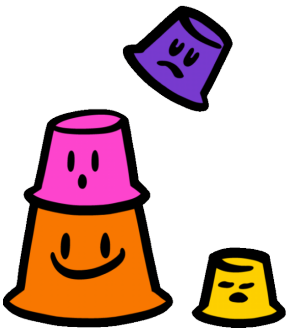
IV. STACK

1. Khái niệm:

Stack (hay còn gọi là ngăn xếp) là một cấu trúc dữ liệu tuyến tính trong đó việc chèn một phần tử mới và loại bỏ một phần tử hiện có diễn ra ở cùng một đầu được biểu thị là đỉnh (top) của ngăn xếp.

Hình ảnh stack cũng rất phổ biến trong cuộc sống hằng ngày, đó là những ngăn xếp mà các vật chồng lên nhau. Khi cần lấy phần tử ra khỏi ngăn xếp thì

phải lấy phần tử đang ở định trước rồi lấy dần xuống. *(Vd: chồng sách, quần áo được xếp trong tủ,...)* → **Vào sau ra trước**



2. Cài đặt:

a. Khai báo:

- Sttack rỗng: stack<***kiểu_dữ_liệu***> ***tên_stack***.
  Ví dụ: stack `<int> st;`

b. Các hàm cơ bản *(Giả định các stack tên st)*

Hàm	Tác dụng	Cài đặt & ví dụ
`size()`	Trả về kích thước của stack→ **O(1)**	`st.size();`
`empty()`	Kiểm tra stack có rỗng không? Nếu có trả về true, nếu không trả về false → **O(1)**	`bool b = st.empty();` `if(!st.empty()) {...}`
`pop()`	Xóa một giá trị khỏi ngăn xếp. Các mục được xuất hiện theo thứ tự đảo ngược khi chúng được đẩy. Nếu ngăn xếp trống thì đó được gọi là tình trạng **Underflow** và thường chạy sinh lỗi.→ **O(1)**	`st.pop();`
`push()`	Thêm một giá trị vào ngăn xếp. Nếu ngăn xếp đầy thì đó được gọi là tình trạng **Overflow**.→ **O(1)**	`st.push(x);`
`top()`	Trả về phần tử trên cùng của ngăn xếp.→ **O(1)**	`cout << st.top();`

`swap()`	Hàm này được sử dụng để hoán đổi nội dung của ngăn xếp này với ngăn xếp khác cùng loại nhưng kích thước có thể khác nhau → **O(1)**	`stack<int> st1, st2;`
		`st1.push(1);`
		`st1.push(2);`
		`st2.push(3);`
		`st2.push(4);`
		`st1.swap(st2);`

c. Sử dụng:

```
int n;
stack <int> st;

//Thêm vào stack n phần tử
cin >> n;
for (int i = 0; i < n; ++i) {
    int x; cin >> x;
    st.push(x);
}

//Duyệt qua các phần tử của stack
while (!st.empty()) {
    cout << st.top() << " ";
    st.pop();
}
```

V. QUEUE

1. Khái niệm:

Queue (hàng đợi) là một cấu trúc dữ liệu dùng để lưu giữ các đối tượng theo cơ chế FIFO (viết tắt từ tiếng Anh: First In First Out), nghĩa là "vào trước ra trước".

Hình ảnh về hàng đợi rất hay gặp trong đời sống hàng ngày, hình ảnh việc xếp hàng dưới đây là một mô phỏng dễ hiểu nhất cho cấu trúc dữ liệu hàng đợi: Người vào đầu tiên sẽ được tiến đón đầu tiên, người mới vào bắt buộc phải xếp hàng ở phía cuối.

Trong cấu trúc hàng đợi (queue), ta chỉ có thể thêm các phần tử vào một đầu của queue (cuối), và cũng chỉ có thể xóa phần tử ở đầu còn lại của queue (đầu). Như vậy, ở một đầu không thể xảy ra hai hành động thêm và xóa đồng thời.

2. Cài đặt:

a. Khai báo: Hàng đợi rỗng: ***queue*** <***kiểu_dữ_liệu***> ***tên_hàng_đợi***.

Ví dụ: `queue <int> q;`

b. Truy cập:

- Truy vấn phần tử tiên: ***tên_hàng_đợi.front()*** → `q.front()`
- Truy vấn phần tử cuối cùng: ***tên_hàng_đợi.back()*** → `q.back()`

c. Các hàm cơ bản: *(Giả định các queue tên **q**)*

Hàm	Tác dụng	Cài đặt
`size()`	Trả về kích thước của hàng đợi → **O(1)**	`q.size();`
`empty()`	Kiểm tra hàng đợi có rỗng hay không? → **O(1)**	`q.empty();` `!q.empty();`
`push()`	Thêm một phần tử ***x*** có cùng KDL vào cuối hang đợi → **O(1)** **đến** **O(n)**	`q.push(x);`
`pop()`	Xóa phần tử đầu của hàng đợi → **O(1)**	`q.pop();`
`emplace()`	Xây dựng và chèn phần tử vào cuối hang đợi → **O(1)** **đến** **O(n)**	`q.emplace();`

V. DEQUE

1. Khái niệm:

Deque là cấu trúc dữ liệu có phần tương tự như queue. Tuy nhiên, deque có thể thêm, bớt và truy cập phần tử ở cả hai đầu chứ không chỉ FIFO (first-in-first-out) như queue.

2. Cài đặt:

a. Khai báo: Hàng đợi rỗng: ***deque*** <***kiểu_dữ_liệu***> ***tên_hàng_đợi***.

Ví dụ: `deque <int> q;`

b. Truy cập các phần tử đầu và cuối tương tự queue. Hoặc có thể truy cặp giống mảng: ***tên_deque [chỉ_số]***

c. Các hàm cơ bản: *(Giả định các deque tên **q**)*

Dưới đây là 4 hàm thao tác với hai đầu của hàng đợi, các hàm còn lại tương tự như queue.

Hàm	Tác dụng	Cài đặt
`push_front()`	Thêm phần tử ***x*** cùng KDL vào đầu hàng đợi → **O(1)**	`q.push_front(x);`
`push_back()`	Thêm phần tử ***x*** cùng KDL vào cuối hàng đợi → **O(1)**	`q.push_back(x);`
`pop_front()`	Xóa phần tử đầu của hàng đợi → **O(1)**	`q.pop_front();`
`pop_back()`	Xóa phần tử cuối của hàng đợi → **O(1)**	`q.pop_back();`

VI. TỔNG KẾT

✏️ Các nội dung nêu trên là các kiểu dữ liệu thường dùng để làm bài. Việc lồng ghép các kiểu dữ liệu với nhau để lưu trữ và truy cập một cách hợp lý chính là "cấu trúc dữ liệu". Học và vận dụng thành thạo CTDL sẽ giúp bạn xử lý nhanh chóng nhiều bài toán có yêu cầu cao về dữ liệu.

✏️ Một số lưu ý khi sử dụng CTDL:

- Không lạm dụng, chỉ dùng khi hiểu rõ tính chất, yêu cầu của chất bài toán và CTDL mà bạn định dùng.
- Nếu dùng CTDL khi không cần thiết sẽ dẫn đến mất thời gian cài đặt và dễ bị rối khi code.
- Cần tập trung và ghi nhớ các địa chỉ lưu trữ của dữ liệu để không bị lúng túng khi truy cập và xử lý.

✏️ Nội dung lần này không có bài tập vận dụng cụ thể, các bạn có thể làm lại những bài tập đã từng làm qua bằng CTDL (chọn bài phù hợp). Vì CTDL đòi hỏi luyện tập nhiều nên ưu tiên code các bài bạn đã nắm rõ cách giải. Bài ***WATATSUMI*** trong contest #5 là một bài vận dung khá ổn về CTDL map và vector.

✏️ Đón chờ contest luyện tập sắp tới tại **GDOJ**.

✏️ Tham gia **đoạn chat cộng đồng** để giao lưu, trao đổi kiến thức, kinh nghiệm cùng các bạn đồng môn và các anh chị học sinh chuyên Tin trên TpHCM.

✏️ Nguồn tham khảo của bài viết: **VNOI**

✏️ Thông tin liên hệ:
- Page: **Căng TIN 2326**
- Group: **Guidelines for Grade 9**
- Gmail: **cangtin2326@gmail.com**

--- Hết ---